

Chapter 4-1

Context-Free Language

PART I: Context-Free Language

Context Free Grammar (CFG)

- A context-free grammar is a tuple $G = (N, T, S, P)$
 - N : a finite set of non-terminals
 - T : a finite set of terminals, such that $N \cap T = \emptyset$
 - $S \in N$: start non-terminals
 - P : production rules in the form of $A \rightarrow a$, where $A \in N$ and $a \in (N \cup T)^*$

assignment $\rightarrow$ identifier = expression
 expression $\rightarrow$ term + term
 term $\rightarrow$ identifier
 term $\rightarrow$ identifier * number

assignment $\rightarrow$ identifier = expression
 expression $\rightarrow$ term + term
 term $\rightarrow$ identifier
 | identifier * number

Derivation ($\Rightarrow_G$)

- Assume: $A \rightarrow \gamma$
- Derivation: $\alpha A \beta \Rightarrow_G \alpha \gamma \beta$, written as $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if G is clear
- Consecutive derivation: $\Rightarrow^*$
- **Example:** $S \rightarrow aSb \mid \epsilon$
 - $S \Rightarrow^* aaabbb$
 - $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaabbb$
- **Context-Free Language:** $L(G) = \{w \in T^*: S \Rightarrow^* w\}$

$a^n b^n$ (not a regular language)

Left-Most Derivation ($\Rightarrow_{lm}$)

- In every step derivation, we replace the left-most non-terminal

- Left-most derivation for $v * (v + d)$

$$\begin{aligned}
 E &\Rightarrow_{lm} EOE \Rightarrow_{lm} vOE \Rightarrow_{lm} v * E \Rightarrow_{lm} v * (E) \Rightarrow_{lm} v * (EOE) \\
 &\Rightarrow_{lm} v * (vOE) \Rightarrow_{lm} v * (v + E) \Rightarrow_{lm} v * (v + d)
 \end{aligned}$$

$E \rightarrow EOE$ $E \rightarrow (E)$ $E \rightarrow v \mid d$ $O \rightarrow + \mid *$
--

- Written as $E \Rightarrow_{lm}^* v * (v + d)$

Right-Most Derivation ($\Rightarrow_{rm}$)

- In every step derivation, we replace the right-most non-terminal

- Right-most derivation for $v * (v + d)$

$$\begin{aligned}
 E &\Rightarrow_{rm} EO\textcolor{red}{E} \Rightarrow_{rm} EO(\textcolor{red}{E}) \Rightarrow_{rm} EO(\textcolor{red}{EOE}) \Rightarrow_{rm} EO(EOd) \Rightarrow_{rm} EO(E + d) \\
 &\Rightarrow_{rm} EO(\textcolor{red}{v} + d) \Rightarrow_{rm} E * (v + d) \Rightarrow_{rm} \textcolor{red}{v} * (v + d)
 \end{aligned}$$

$E \rightarrow EOE$ $E \rightarrow (E)$ $E \rightarrow v \mid d$ $O \rightarrow + \mid *$
--

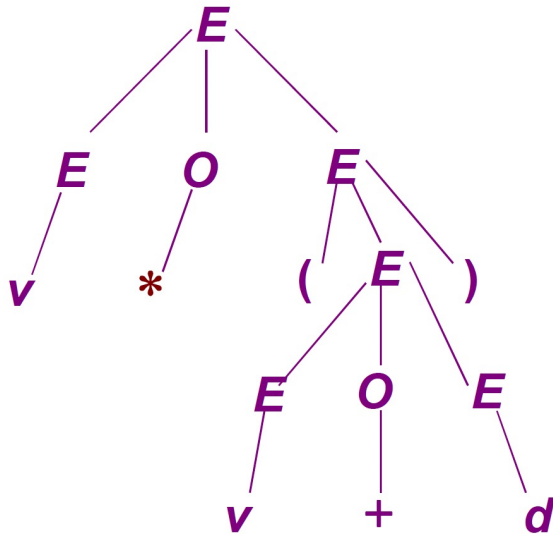
- Written as $E \Rightarrow_{rm}^* v * (v + d)$

Parse/Syntax Trees

- Derivation is a procedure of building a parse tree

$$\begin{aligned}
 E &\Rightarrow_{lm} EOE \Rightarrow_{lm} vOE \Rightarrow_{lm} v * E \Rightarrow_{lm} v * (E) \Rightarrow_{lm} v * (EOE) \\
 &\Rightarrow_{lm} v * (vOE) \Rightarrow_{lm} v * (v + E) \Rightarrow_{lm} v * (v + d)
 \end{aligned}$$

$E \rightarrow EOE$
$E \rightarrow (E)$
$E \rightarrow v \mid d$
$O \rightarrow + \mid *$



- Parse tree:
 - Every leaf is a terminal
 - Every non-leaf node is a non-terminal

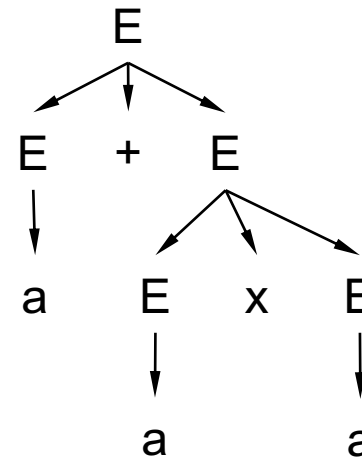
Ambiguity

- $E \rightarrow E + E \mid E \times E \mid a$
- $w = a + a \times a$

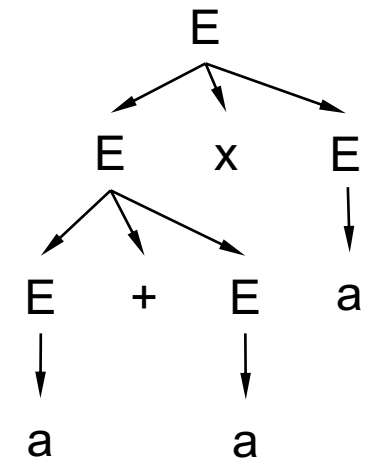
$E \Rightarrow E + E \Rightarrow E + E \times E \Rightarrow \dots \Rightarrow w$

$E \Rightarrow E \times E \Rightarrow E + E \times E \Rightarrow \dots \Rightarrow w$

$$2 + 2 \times 2 = 6$$



$$2 + 2 \times 2 = 8$$



- A grammar is ambiguous if there is a string has two different derivation trees
- A grammar is not ambiguous if for any string, it has only one derivation tree

Inherent Ambiguity

- Some context-free languages have only ambiguous grammars
- We cannot eliminate the ambiguity
- There is not a general technique to eliminate the ambiguity!
- There is not an algorithm that can decide if a grammar is ambiguous (This is an undecidable problem!)

Eliminating Ambiguity

- Lifting operators with higher priority

- $E \rightarrow E + E \mid E \times E \mid a$

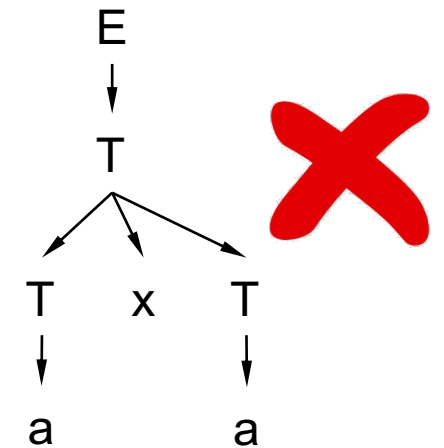
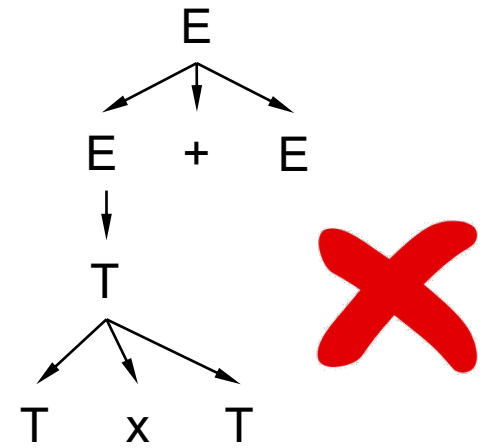
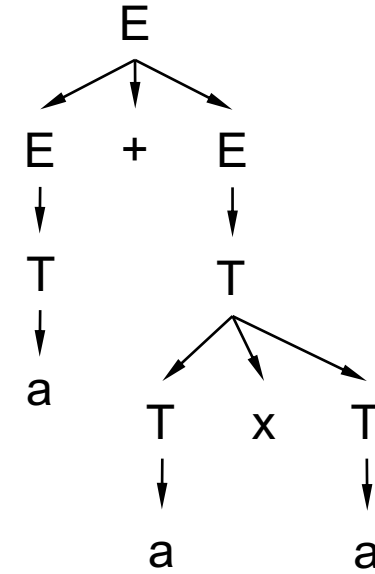
- -----

- $E \rightarrow E + E \mid T$

- $T \rightarrow T \times T \mid a$

- -----

- $w = a + a \times a$



$$E \Rightarrow E + E \Rightarrow E + T \times T \Rightarrow T + T \times T \Rightarrow a + a \times a$$

$$E \Rightarrow E + E \Rightarrow T \times T + E \Rightarrow \dots \text{ (does not work)}$$

$$E \Rightarrow T \Rightarrow T \times T \Rightarrow \dots \text{ (does not work)}$$

Eliminating Ambiguity

- $E \rightarrow E + E \mid E - E \mid T;$

- $T \rightarrow T \times T \mid a \mid b$

- -----

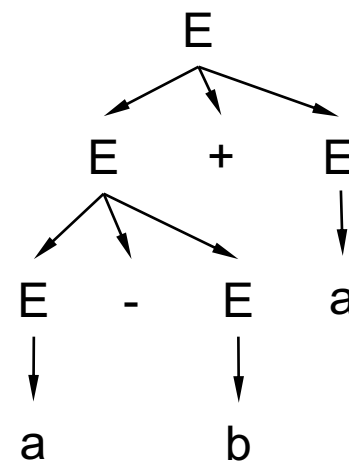
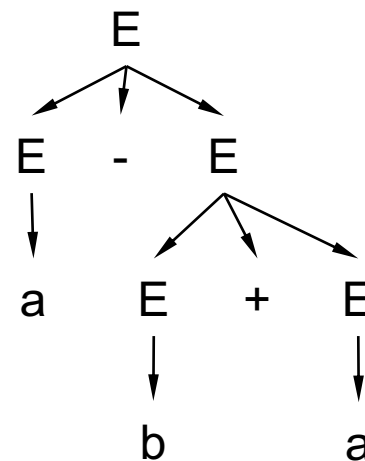
- $w = a - b + a$

- -----

- Left associativity for most operators of the same priority

- -----

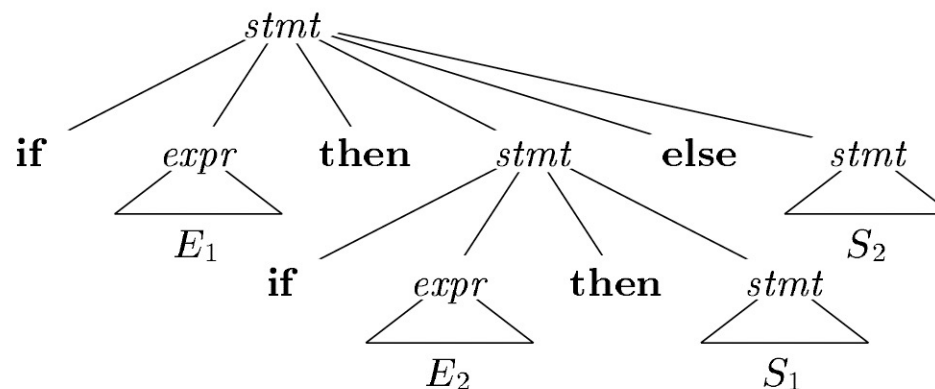
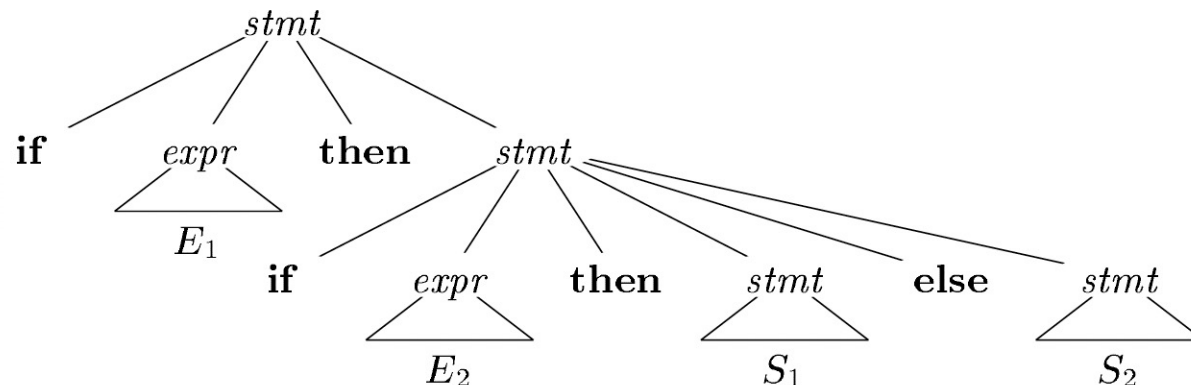
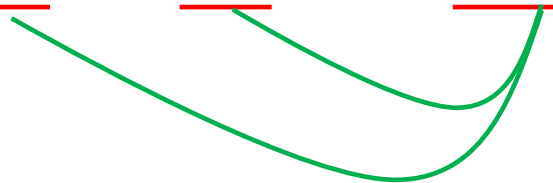
- $E \rightarrow E + T \mid E - T \mid T; \quad T \rightarrow T \times T \mid a \mid b$



Eliminating Ambiguity

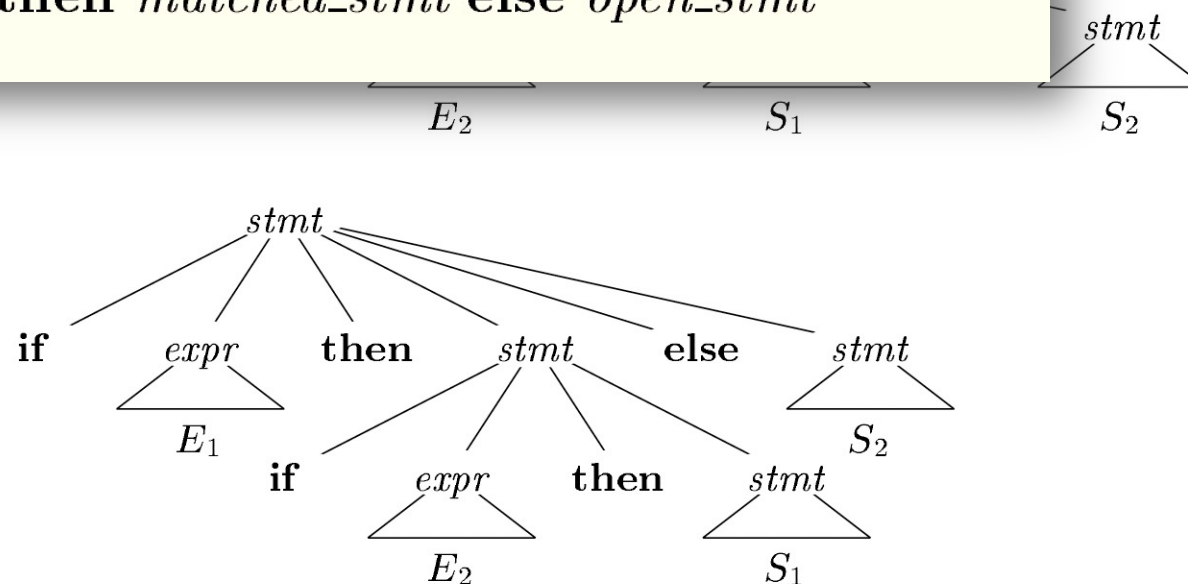
stmt → if *expr* then *stmt*
| if *expr* then *stmt* else *stmt*
| other

if E_1 then if E_2 then S_1 else S_2



Eliminating Ambiguity

<i>stmt</i>	→	<i>matched_stmt</i>
		<i>open_stmt</i>
<i>matched_stmt</i>	→	if <i>expr</i> then <i>matched_stmt</i> else <i>matched_stmt</i>
		other
<i>open_stmt</i>	→	if <i>expr</i> then <i>stmt</i>
		if <i>expr</i> then <i>matched_stmt</i> else <i>open_stmt</i>



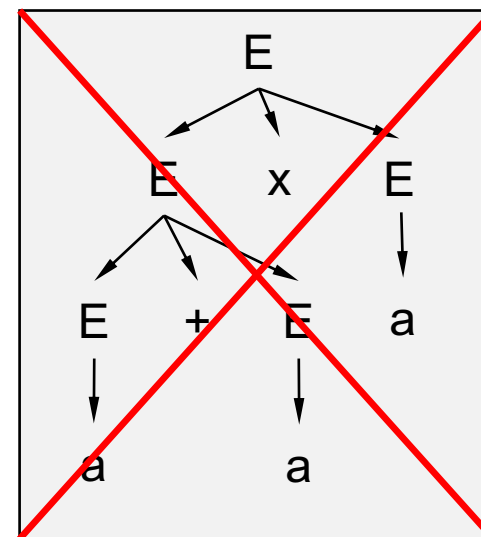
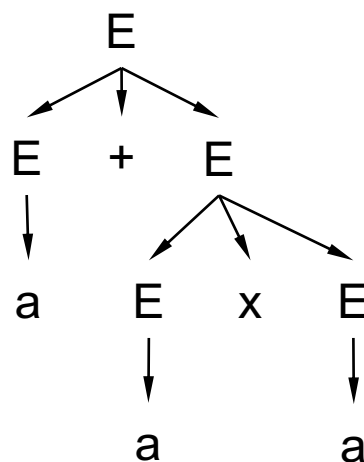
Using Ambiguous Grammar

- Given that we cannot always eliminate ambiguity, just use it!
- We will discuss how we use ambiguous grammar in the next lecture when introducing specific parsing techniques

- $E \rightarrow E + E \mid E \times E \mid a$

- $w = a + a \times a$

Let us always try production rules in order!



Exercises

$S \rightarrow aSb \mid \epsilon$ is the grammar for $a^n b^n$

- Write a context-free grammar for the following languages
 - 0^*1^*
 - 0^n1^{2n}
 - $L = \{ w \mid w \text{ is a string of balanced parentheses} \}$
 - $a^i b^j c^k$ where $i = j$ or $j = k$
 - $a^i b^j c^k$ where $i \neq j$ or $j \neq k$

Exercises

- Prove all regular languages are context-free languages

Primitive regex

- $\emptyset$,
- ϵ
- a

Primitive regex

- $S \rightarrow S$
- $S \rightarrow \epsilon$
- $S \rightarrow a$

Given two regex: r_1, r_2 , the following are regex

- $r_1 | r_2$
- $r_1 r_2$
- r_1^*
- (r_1)

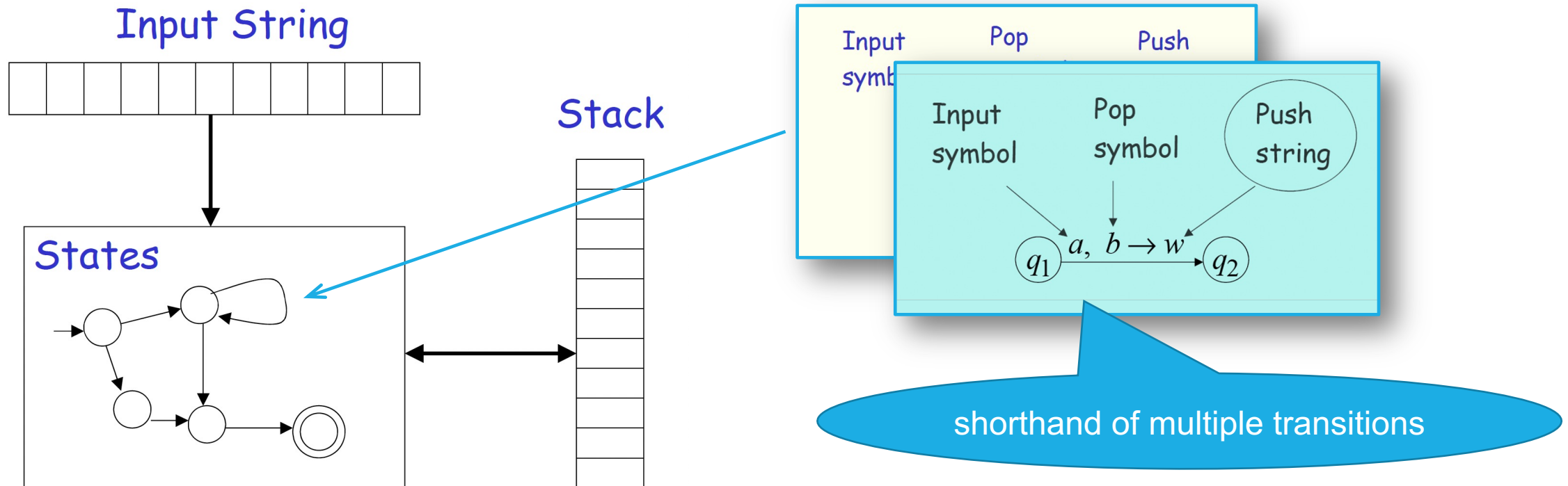
Given the CFG start symbols S_1, S_2 , we have

- $S \rightarrow S_1 | S_2$
- $S \rightarrow S_1 S_2$
- $S \rightarrow S_1 S | \epsilon$
- $S \rightarrow (S_1)$

PART II: Push-Down Automata

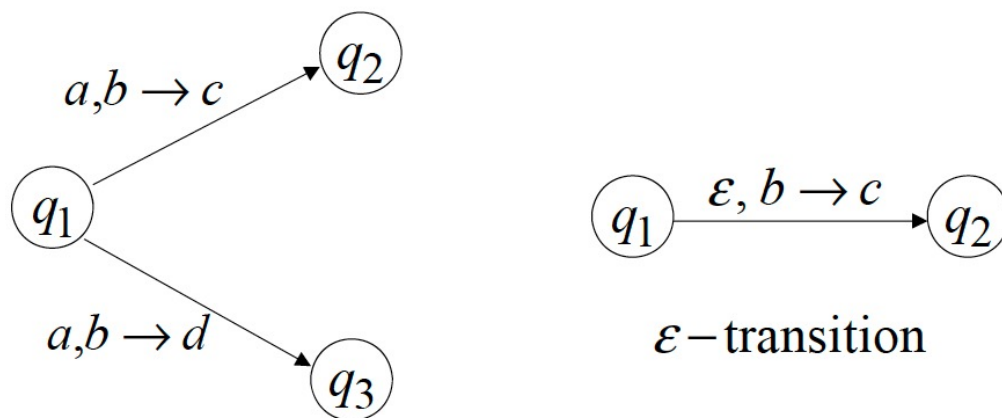
Push-Down Automata (PDA)

- Regular language = DFA/NFA
- Context-free language = PDA = **NFA + Stack (z_0) = NPDA**



Non-Deterministic PDA (NPDA)

- Regular language = DFA/NFA
- Context-free language = PDA = NFA + Stack (z_0) = NPDA

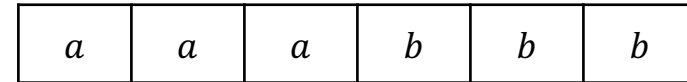
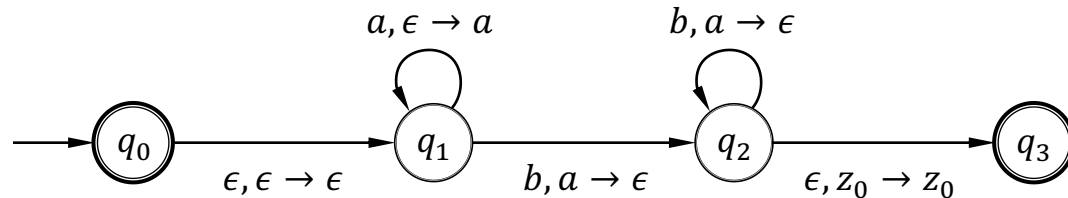


Non-Deterministic PDA (NPDA)

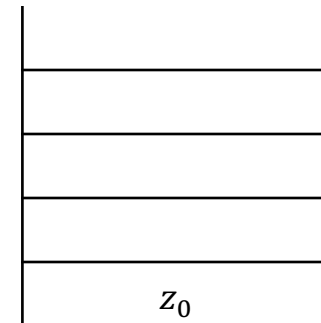
- NPDA/PDA is defined by the septuple: $P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$
 - Q : a finite set of states
 - Σ : finite input alphabet
 - Γ : finite stack alphabet
 - $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \mapsto 2^{Q \times \Gamma^*}$: transition
 - $q_0 \in Q$: initial state
 - $z_0 \in \Gamma$: stack start symbol
 - $F \subseteq Q$: set of final states

Non-Deterministic PDA (NPDA)

- Recall that $a^n b^n$ is not regular and cannot be accepted by DFA
- Strings in the language can be accepted by the following NPDA

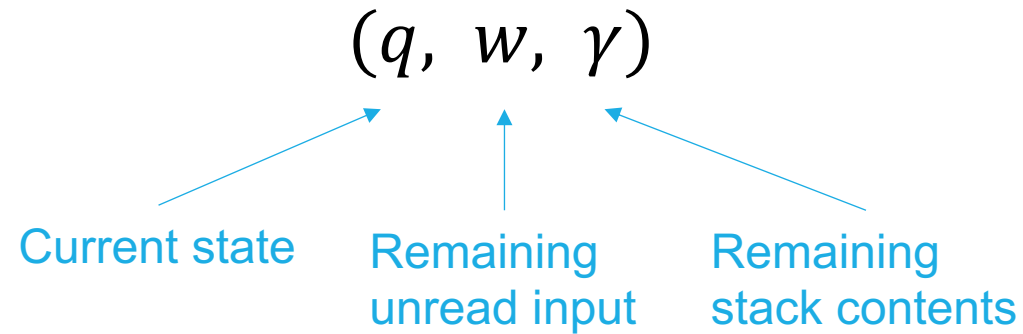


- Let's check how it accepts the string $aaabbb$



Instantaneous Description

- An instantaneous description of a PDA is a triple

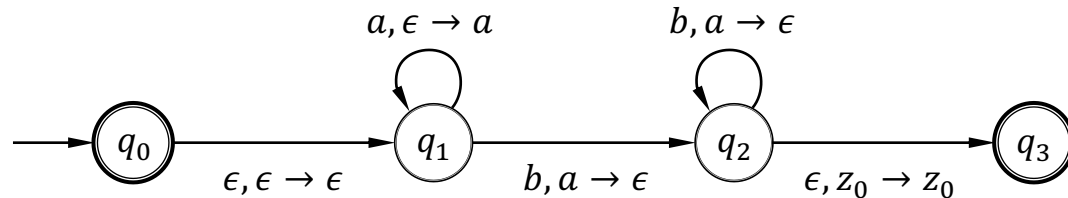


- If $(q, \alpha) \in \delta(p, a, X)$ then $(p, \textcolor{red}{a}w, \textcolor{blue}{X}\beta) \vdash_M (q, \textcolor{red}{w}, \textcolor{blue}{\alpha}\beta)$

Write $\vdash$ when M is clear

Instantaneous Description

- Recall that $a^n b^n$ is not regular and cannot be accepted by DFA
- Strings in the language can be accepted by the following NPDA



- Let's check how it accepts the string $aaabbb$

$(q_0, aaabbb, z_0) \vdash (q_1, aaabbb, z_0) \vdash (q_1, aabbb, az_0) \vdash (q_1, abbb, aaz_0) \vdash (q_1, bbb, aaaz_0)$
 $\vdash (q_2, bb, aaz_0) \vdash (q_2, b, az_0) \vdash (q_2, \epsilon, z_0) \vdash (q_3, \epsilon, z_0)$

Language of PDA

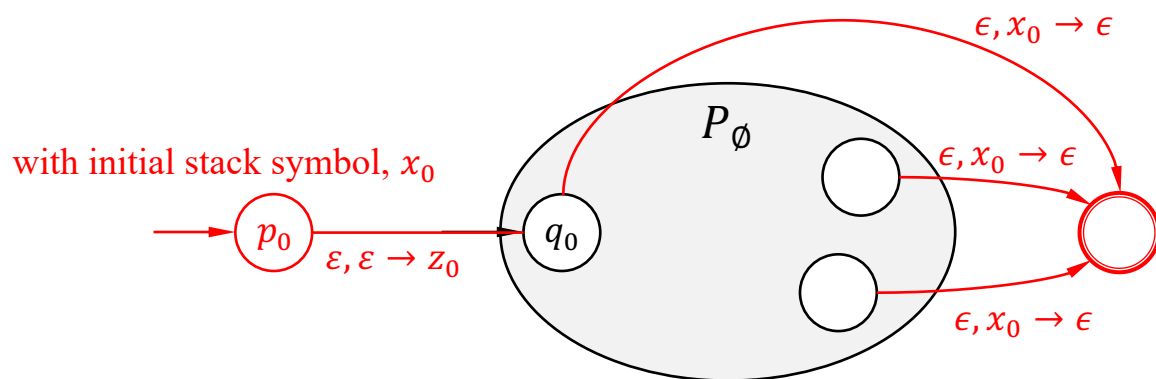
- A string is accepted if there is a computation such that
 - (1) all the input symbols are consumed
 - (2) the last state is a final state
- At the end of computation, we don't care about stack contents
- The language of a PDA is the set of strings accepted by it
 - $L(M) = \{w \in \Sigma^* : (q_0, w, z_0) \vdash_M^* (q_f, \epsilon, u), q_f \in F, u \in \Gamma^*\}$
 - $L(M) = \{w \in \Sigma^* : (q_0, w, z_0) \vdash_M^* (q, \epsilon, \epsilon), q \in Q\}$

Equivalent definitions

- (1) acceptance by final states
- (2) acceptance by empty stack

Language of PDA

- The language of a PDA is the set of strings accepted by it
 - $L(M) = \{w \in \Sigma^* : (q_0, w, z_0) \vdash_M^* (q_f, \epsilon, u), q_f \in F, u \in \Gamma^*\}$
 - $L(M) = \{w \in \Sigma^* : (q_0, w, z_0) \vdash_M^* (q, \epsilon, \epsilon), q \in Q\}$
- From empty stack ($P_\emptyset$) to final states (P_F)

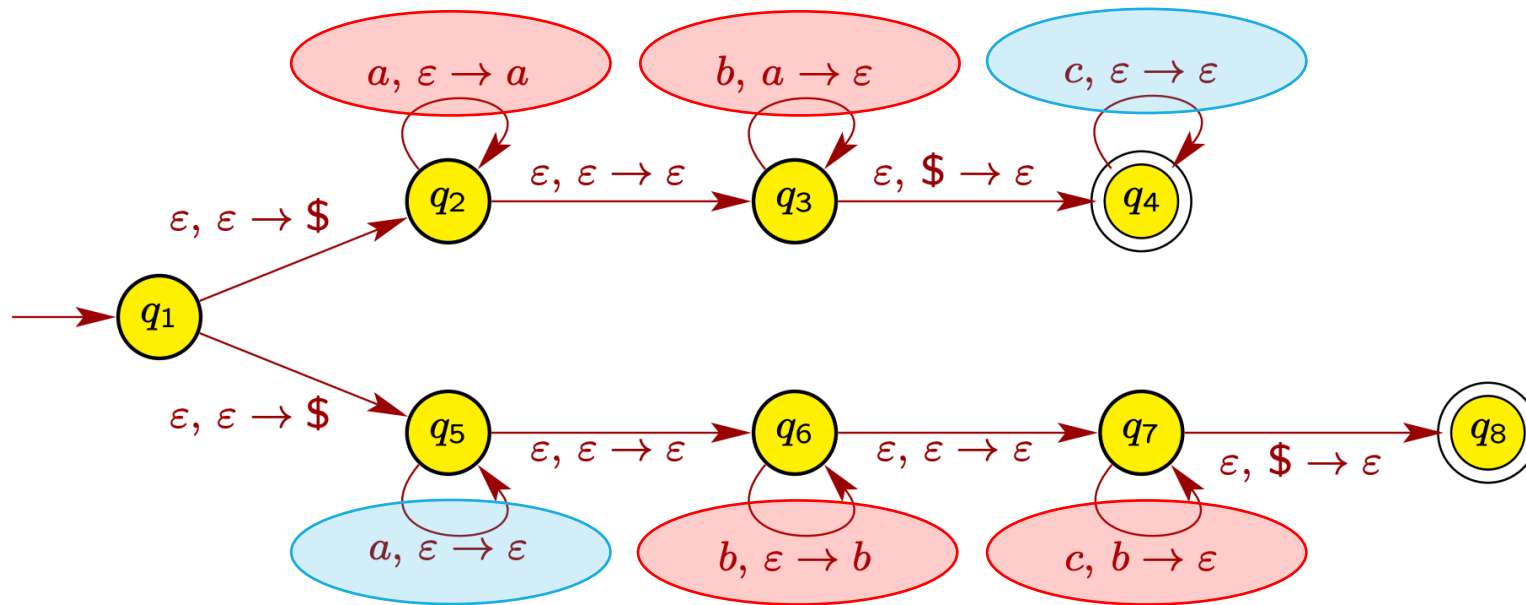


Reaching Final States $\Rightarrow$ Reaching Empty Stack

Reaching Final States $\Leftarrow$ Reaching Empty Stack

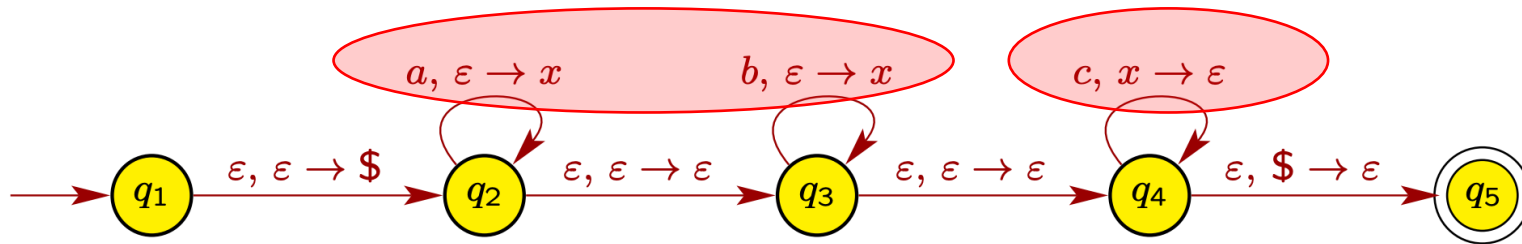
Exercises

- Design a PDA to accept $a^i b^j c^k$ where $i = j$ or $j = k$, and write the formal definition of the PDA



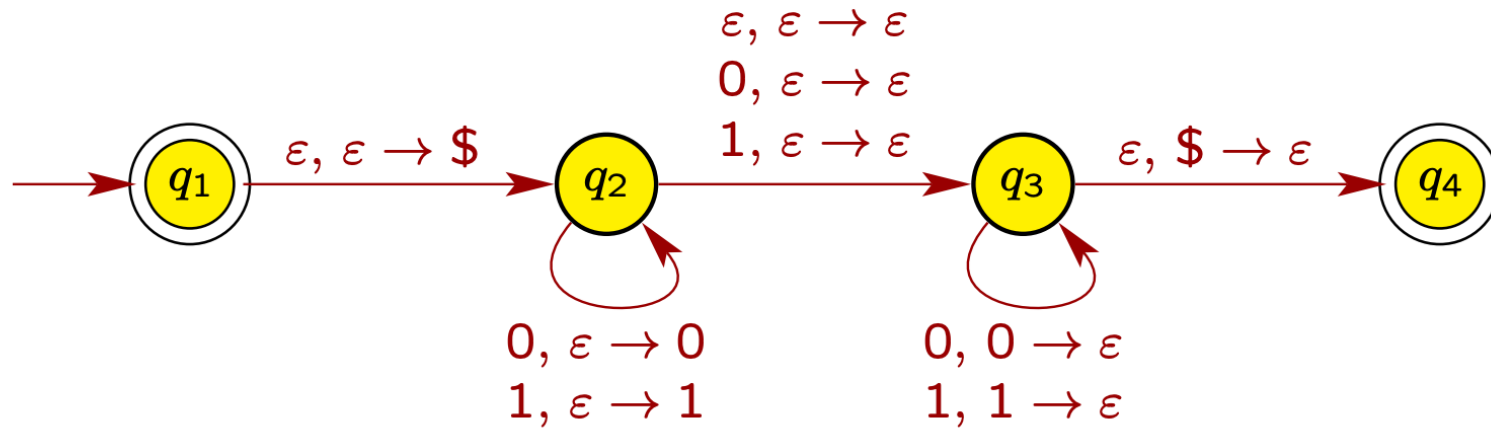
Exercises

- Design a PDA to accept $a^i b^j c^k$ where $i + j = k$, and write the formal definition of the PDA



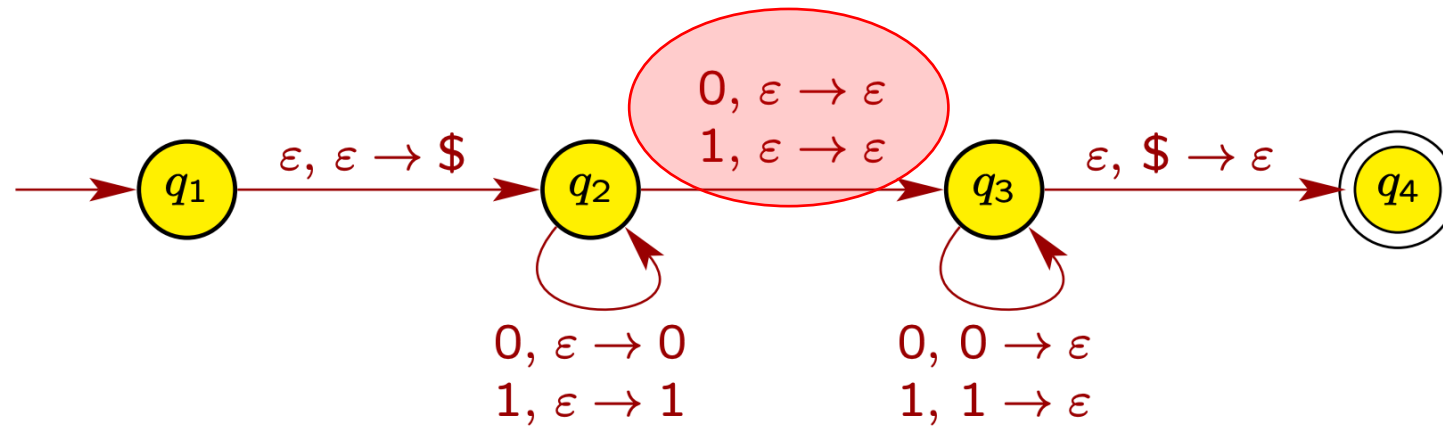
Exercises

- Design a PDA to accept $w \in \{0,1\}^*$ where $w = w^R$. Write the formal definition of the PDA



Exercises

- Design a PDA to accept $w \in \{0,1\}^*$ where $w = w^R$ and $|w|$ is odd. Write the formal definition of the PDA



Exercises

- Prove that, for any PDA, there is an equivalent PDA consisting of only two stack symbols

PART III: CFG = PDA

Equivalence of PDA and CFG

- $\text{CFG} \subseteq \text{PDA}$
- $\text{PDA} \subseteq \text{CFG}$

CFG $\subseteq$ PDA

Exercise: Try to prove the equivalence!

- Given any CFG, $G = (N, T, P, S)$, we can build an equivalent PDA that accepts string by empty stack, $(\{q\}, T, N \cup T, \delta, q, S, \emptyset)$, where
 - (1) for any non-terminal $A \in N$, $\delta(q, \epsilon, A) = \{(q, \beta) : A \rightarrow \beta \in P\}$
 - (2) for any terminal $a \in T$, $\delta(q, a, a) = \{(q, \epsilon)\}$
- Example:** $S \rightarrow aSb \mid \epsilon$, the grammar for $a^n b^n$. Check with $aabb$

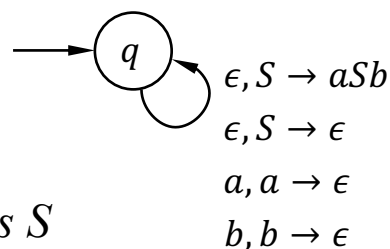


PDA $\subseteq$ CFG

Exercise: Try to prove the equivalence!

- Given any PDA, $(Q, \Sigma, \Gamma, \delta, q_0, z_0, \emptyset)$, we can build an equivalent CFG (N, Σ, P, S) , where $N = \{S\} \cup \{N_{pXq} : p, q \in Q, X \in \Gamma\}$
- The production rules are defined below, where $a \in \Sigma \cup \{\epsilon\}$
 - (1) $\forall p \in Q : S \rightarrow N_{q_0 z_0 p} \in P$
 - (2) $(q, \epsilon) \in \delta(p, a, X) \Rightarrow N_{pXq} \rightarrow a \in P$
 - (3) $(q, X_1 X_2 \cdots X_k) \in \delta(p, a, X) \Rightarrow N_{pXp_k} \rightarrow a N_{qX_1 p_1} N_{p_1 X_2 p_2} \cdots N_{p_{k-1} X_k p_k} \in P$

Example:



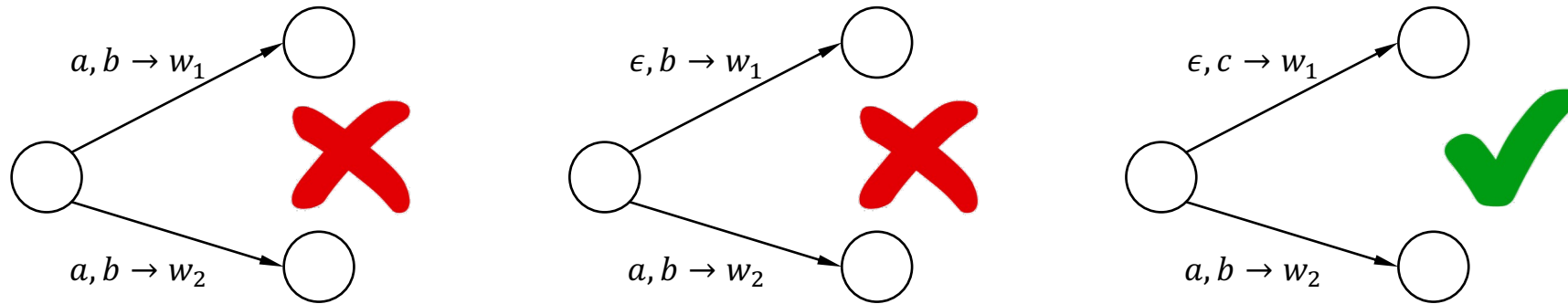
initial stack symbol is S

$S \rightarrow N_{qSq}$
 $N_{qSq} \rightarrow \epsilon$
 $N_{qaq} \rightarrow a$
 $N_{q bq} \rightarrow b$
 $N_{qSq} \rightarrow \epsilon N_{qaq} N_{qSq} N_{q bq} \rightarrow a N_{qSq} b$

$S \rightarrow N_{qSq}$
 $N_{qSq} \rightarrow \epsilon$
 $N_{qSq} \rightarrow a N_{qSq} b$
 $S \rightarrow \epsilon$
 $S \rightarrow aSb$

Deterministic PDA (DPDA)

- $\text{DFA/NFA} \subseteq \text{DPDA} \subseteq \text{NPDA}$



- A CFL must have a NPDA, but may not have a DPDA
- A DPDA language has a CFG without ambiguity

PART IV: Properties of CFL

Closure Properties

- Given two CFL, L_1 and L_2 , the following are CFL
 - $L_1 \cup L_2$
 - $L_1 L_2$
 - L_1^*
 - L_1^R

Intersection of CFL

Assume $\{a^n b^n c^n\}$ is not a CFL

- Given two CFL, L_1 and L_2 , the following may not be CFL
 - $L_1 \cap L_2$
 - $\overline{L_1}$
 - $L_1 - L_2$

Intersection of CFL

Assume $\{a^n b^n c^n\}$ is not a CFL

- Given a CFL L_1 and RL L_2 , $L_1 \cap L_2$ is CFL
- **Proof:** Construct a new NPDA that simulates the PDA of L_1 and the DFA of L_2 in parallel

Application of the Properties

- Prove a language is a CFL or not

Application of the Properties

- Prove that $L = \{a^n b^n : n \neq 100, n \geq 0\}$ is a CFL
- **Proof:**
 - $L_1 = \{a^{100} b^{100}\}$ is regular
 - $\overline{L_1} = \{a, b\}^* - \{a^{100} b^{100}\}$ is regular
 - $L = \{a^n b^n\} \cap \overline{L_1}$ is context-free as we know $\{a^n b^n\}$ is context-free

Application of the Properties

- Prove that $L = \{w \in \{a, b, c\}^* : n_a = n_b = n_c\}$ is not a CFL
- **Proof:**
 - Assume L is context-free
 - $L \cap \{a^*b^*c^*\}$ is context free as the former is CFL, the latter is RL
 - However, $L \cap \{a^*b^*c^*\} = \{a^n b^n c^n\}$, which, as we know, is not a CFL
 - The assumption is wrong

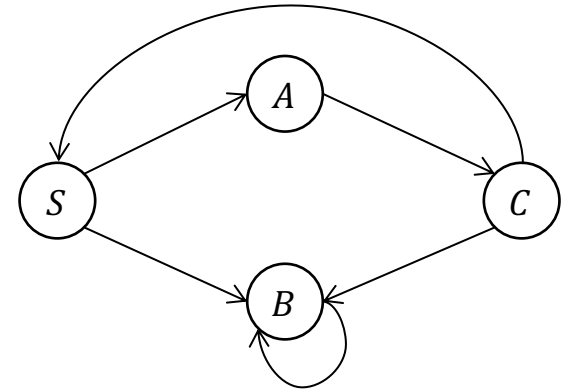
Decidable Properties

- **Empty Language Question:**
 - Given a CFG, is the CFL empty?
- **Algorithm:**
 - Check if the start non-terminal is not used, or useless, e.g., $S \rightarrow S$

Decidable Properties

- **Infinite Language Question:**

- Given a CFG, is the CFL infinite?

$$\begin{array}{l} S \rightarrow AB \\ A \rightarrow aCb \mid a \\ B \rightarrow bB \mid bb \\ C \rightarrow cBS \end{array}$$


- **Algorithm:**

- Remove useless non-terminals
- Remove unit and epsilon productions
- Create a dependency graph for remaining non-terminals
- Check if the graph has a circle

Decidable Properties

- **Membership Question:**
 - Given a CFG, is a string belongs to the CFL?
- **Algorithm 1:**
 - Create an NPDA of the CFG
 - Check if the NPDA can accept the string
- **Algorithm 2:**
 - the CYK algorithm ($O(n^3)$)
 - https://en.wikipedia.org/wiki/CYK_algorithm

We will come back to this (CFL-reachability) question when introducing the middle end of compilers

Undecidable Properties

- Check if a CFG is not ambiguous
- Check if a CFG has inherent ambiguity
- Check if the intersection of two CFLs is empty
- Check if two CFLs are equivalent
- Check if a CFL is equivalent to Σ^*

PART V: Pumping Lemma for CFL

Chomsky Normal Form

- We can rewrite a CFG in many standard forms, e.g., CNF
- A CFG is in CNF if all its production rules are of the form
 - $A \rightarrow BC$; $A \rightarrow a$; $S \rightarrow \epsilon$
- A , B , and C are non-terminals and a is a terminal
- S is the start symbol, B and C cannot be start symbol
- $S \rightarrow \epsilon$ can appear if ϵ is in the language
- https://en.wikipedia.org/wiki/Chomsky_normal_form

Pumping Lemma for CFL

- Given a CFL L , there exists a constant n such that
- we have $z = uvwxy \in L, |z| \geq n$ satisfying the conditions below
 - $vx \neq \epsilon$
 - $|vwx| \leq n$
 - $\forall i \geq 0, z_i = uv^iwx^iy \in L$
- To prove a language is not a CFL, show that we cannot split a string in such a manner.

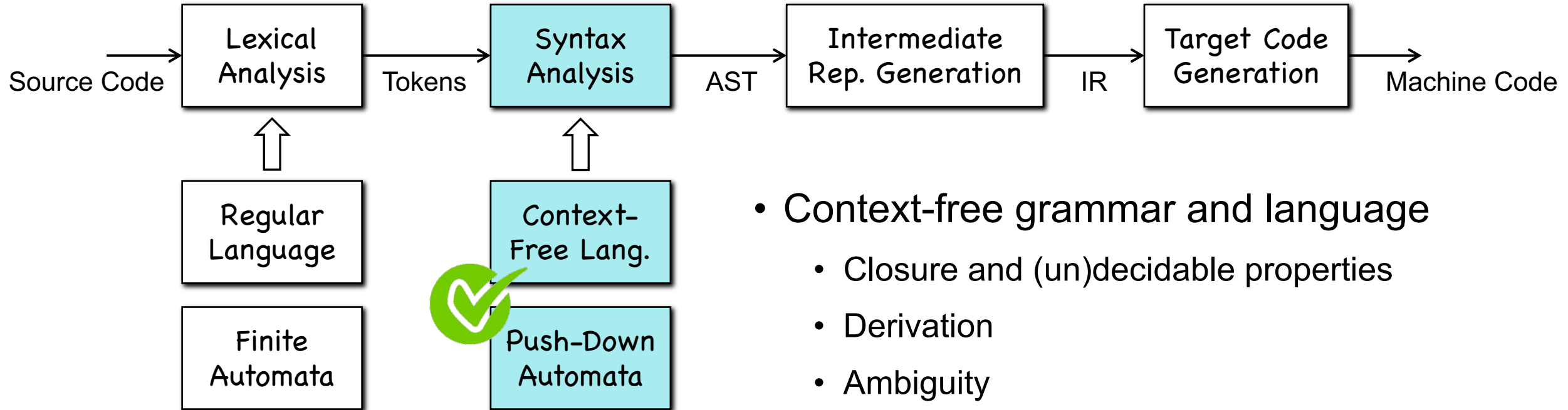
Proving $L = \{a^n b^n c^n\}$ is not a CFL

- Assume it is a CFL, we then can split it as
- $z = a^n b^n c^n = uvwxy$, where $|z| \geq n \wedge |vwx| \leq n$
- **Case 1:** vwx consists of only a , pumping v, x will increase $\#a$
- **Case 2:** vwx consists of only b , pumping v, x will increase $\#b$
- **Case 3:** vwx consists of only c , pumping v, x will increase $\#c$
- **Case 4:** vwx consists of a, b , pumping v, x will increase $\#a, b$
- **Case 5:** vwx consists of b, c , pumping v, x will increase $\#b, c$

Proving $L = \{a^{n^2}\}$ is not a CFL

- Assume it is a CFL, we then can split it as
- $z = a^{n^2} = uvwxy$, where $|z| \geq n \wedge |vwx| \leq n \wedge 1 \leq |vx| \leq n$
- Let us pump vx twice, we have $z_2 = uv^2wx^2y$, where
 - $|z_2| = n^2 + |vx|$
 - $n^2 + 1 \leq |z_2| \leq n^2 + n < (n + 1)^2$

Summary



- Context-free grammar and language
 - Closure and (un)decidable properties
 - Derivation
 - Ambiguity
- PDA vs. NPDA vs. DPDA vs. CFL
- The Pumping Lemma for CFL

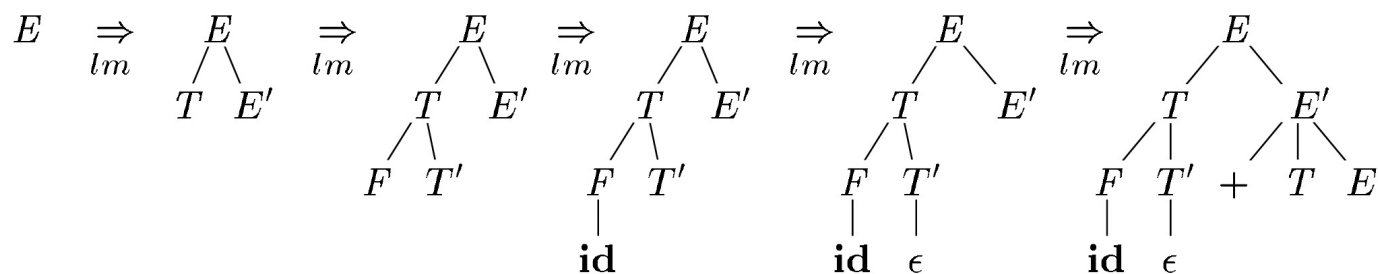
Chapter 4-2

Syntax Analysis

PART I: Top-Down Parsing

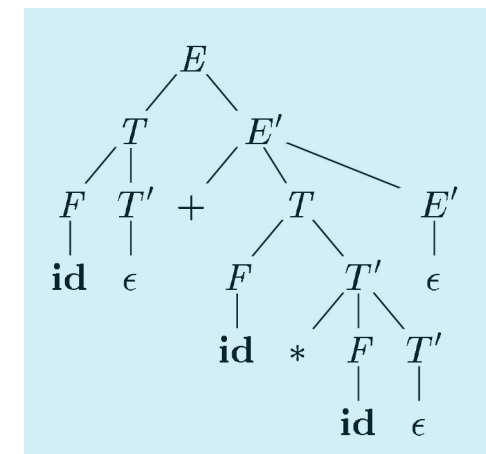
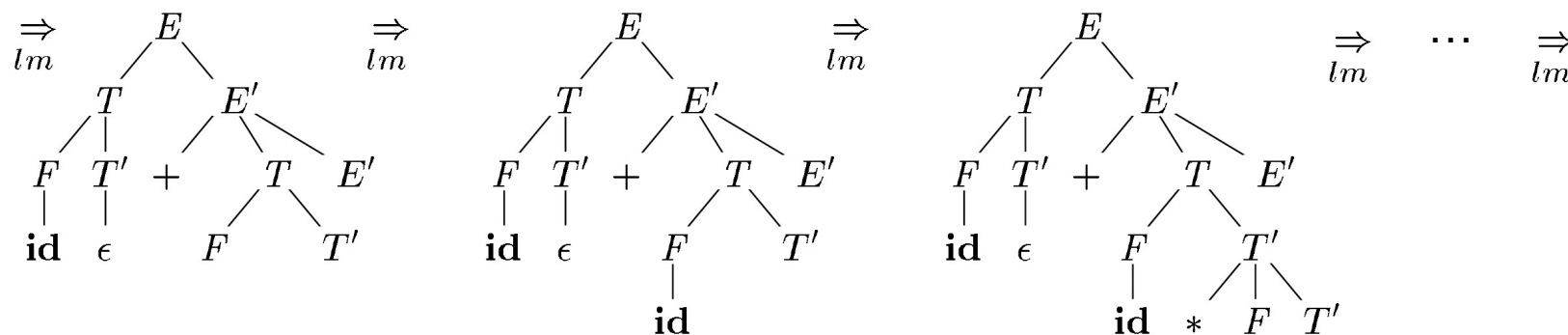
Top-Down Parsing

- Top-down parsing can be viewed as the problem of constructing a parse tree in **preorder**



E	$\rightarrow$	$T E'$
E'	$\rightarrow$	$+ T E' \mid \epsilon$
T	$\rightarrow$	$F T'$
T'	$\rightarrow$	$* F T' \mid \epsilon$
F	$\rightarrow$	$(E) \mid \text{id}$

id + id * id



Building a Parser in Practice

- Each non-terminal has a procedure or function for parsing
- => Recursive-Descent Parser
- **Example:** $S \rightarrow c A b$; $A \rightarrow a b \mid a$



Let's parse

c	a	b
---	---	---

```
bool S() {
    if (*cursor == 'c') cursor++;
    else return false;
    if (!A()) return false;
    if (*cursor == 'b') cursor++;
    else return false;    ???
    return true;
}
```

```
bool A() {
    temp = cursor;
    cursor = temp;
    if (*cursor == 'a') {
        cursor++;
        if (*cursor == 'b') return true;
    }

    cursor = temp;
    if (*cursor == 'a') return true;
    return false;
}
```


Building a Parser in Practice

- Each non-terminal has a procedure or function for parsing
- $\Rightarrow$ Recursive-Descent Parser
- **Problem 1:** Backtracking may be necessary
 - when one derivation does not work, we may try others
- **Problem 2:** A left-recursive grammar can cause infinite loops
 - when expanding a non-terminal, we may find itself and expand it again
- **Example:** $A \rightarrow A b \mid a$

Eliminating Left-Recursion

- A grammar is left-recursive if it has a non-terminal A such that there is a derivation $A \Rightarrow^+ A\alpha$
- **Example:** $A \rightarrow A\alpha \mid \beta$ is left-recursive, can be transformed into

$$\begin{array}{l} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \epsilon \end{array}$$

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

$$\begin{array}{l} A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A' \\ A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \epsilon \end{array}$$

Eliminating Left-Recursion

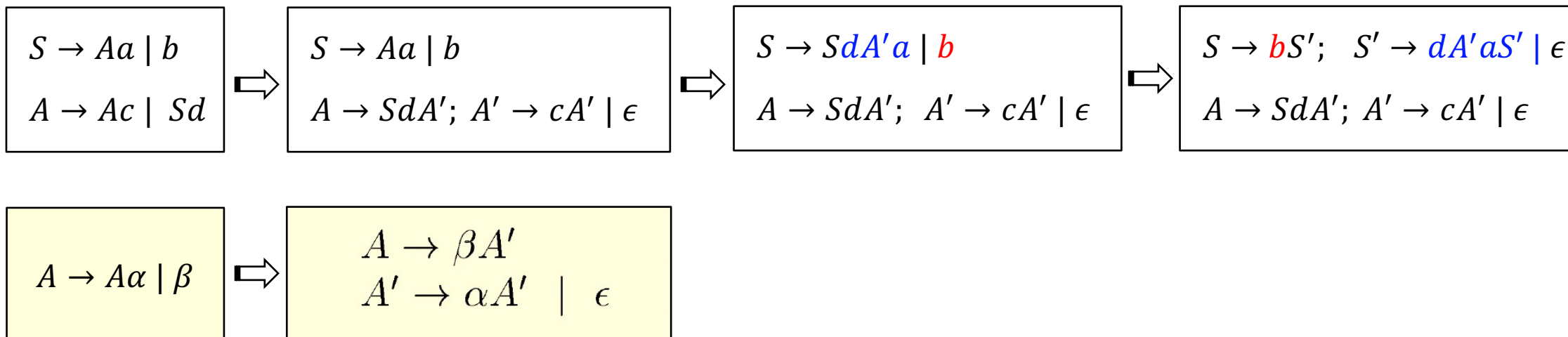
- 1) arrange the nonterminals in some order $A_1, A_2, \dots, A_n$.
- 2) **for** (each i from 1 to n) {
- 3) **for** (each j from 1 to $i - 1$) {
- 4) replace each production of the form $A_i \rightarrow A_j \gamma$ by the
 productions $A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$, where
 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ are all current A_j -productions
- 5) }
- 6) eliminate the immediate left recursion among the A_i -productions
- 7) }

Any grammar can be converted to a grammar that does NOT include
 (1) cycles ($A \Rightarrow^+ A$) or (2) ϵ productions*

** with possible exception of the empty string*

Eliminating Left-Recursion

• Example



Building a Parser in Practice

- Each non-terminal has a procedure or function for parsing
- => Recursive-Descent Parser
- **Problem 1:** Backtracking may be necessary
 - when one derivation does not work, we may try others
-  **Problem 2:** A **left-recursive** grammar can cause **infinite loops**
 - when expanding a non-terminal, we may find itself and expand it again

Predictive Parsing

- Predictive parsers are recursive descent parser w/o backtracking
- LL(1)
 - L: scanning input from left to right
 - L: leftmost derivation
 - 1: Using one input symbol of lookahead at each step
- LL(1) grammar (Not ambiguous! Not left-recursive!)
 - **Rich enough** to cover most programming constructs

Predictive Parsing

• Predictive

• LL(1)

• L: S

• L: E

• 1: T

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

backtracking

• LL(1) grammar (Not ambiguous! Not left-recursive!)

- **Rich enough** to cover most programming constructs
- To build the predictive table, let's define **FIRST(α)**; **FOLLOW(α)**

First() and Follow()

- $\text{FIRST}(\alpha)$:
 - A set of terminals that α may start with
- $\text{FOLLOW}(\alpha)$:
 - A set of terminals that can appear immediately to the right of α

First()

- $\text{FIRST}(\alpha)$:
 - A set of terminals that α may start with
 - If X is a terminal, $\text{FIRST}(X) = \{X\}$
 - If $X \rightarrow \epsilon$ is a production, $\epsilon \in \text{FIRST}(X)$
 - If $X \rightarrow Y_1 Y_2 \dots Y_k$, $\epsilon \in \bigcap_{j=1}^{i-1} \text{FIRST}(Y_j) \wedge a \in \text{FIRST}(Y_i) \Rightarrow a \in \text{FIRST}(X)$
 $\epsilon \in \bigcap_{j=1}^k \text{FIRST}(Y_j) \Rightarrow \epsilon \in \text{FIRST}(X)$

$$X \rightarrow Y_1 Y_2 \dots Y_k \rightarrow \epsilon$$

First()

- **Example:** $S \rightarrow c A b; A \rightarrow a b \mid a$
- $\text{FIRST}(S) = \{c\}$
- $\text{FIRST}(A) = \{a\}$
- $\text{FIRST}(a) = \{a\}$
- $\text{FIRST}(b) = \{b\}$
- $\text{FIRST}(c) = \{c\}$

First()

- **Exercise:** write First() for all symbols in the following grammar

$E \rightarrow T X$
 $X \rightarrow + E$
 $X \rightarrow \varepsilon$
 $T \rightarrow \text{int } Y$
 $T \rightarrow (E)$
 $Y \rightarrow * T$
 $Y \rightarrow \varepsilon$

Symbol	First
(	(
)	)
+	+
*	*
int	int
Y	$\varepsilon, *$
X	$\varepsilon, +$
T	int, (
E	int, (

Follow()

- FOLLOW(α):
 - A set of terminals that can appear immediately to the right of α
 - $\$ \in \text{FOLLOW}(S)$, where $\$$ is string's end marker, S the start non-terminal
 - $A \rightarrow \alpha B \beta \Rightarrow \text{FIRST}(\beta) \setminus \{\epsilon\} \subseteq \text{FOLLOW}(B)$
 - $A \rightarrow \alpha B$ or $A \rightarrow \alpha B \beta$ where $\epsilon \in \text{FIRST}(\beta) \Rightarrow \text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$

$A \text{ or } \alpha B$	...
--------------------------	-----

- **Note: repeat the procedure until fixed point!**

Follow()

- **Example:** $S \rightarrow c A b; A \rightarrow a b \mid a$
- $\text{FOLLOW}(S) = \{\$ \}$
- $\text{FOLLOW}(A) = \{b\}$
- **Example:** $S \rightarrow c A A; A \rightarrow a b \mid a$
- $\text{FOLLOW}(S) = \{\$ \}$
- $\text{FOLLOW}(A) \supseteq \text{FIRST}(A) \setminus \{\epsilon\} = \{a\}$
- $\text{FOLLOW}(A) \supseteq \text{FOLLOW}(S) = \{\$ \}$

Follow()

- **Exercise:** write Follow() for all symbols in the grammar

$E \rightarrow T X$
 $X \rightarrow + E$
 $X \rightarrow \varepsilon$
 $T \rightarrow \text{int } Y$
 $T \rightarrow (E)$
 $Y \rightarrow * T$
 $Y \rightarrow \varepsilon$

Symbol	First	Follow
(	(	N/A
)	)	
+	+	
*	*	
int	int	
Y	$\varepsilon, *$	), \$, +
X	$\varepsilon, +$	), \$
T	int, (	), \$, +
E	int, (	), \$

Predictive Parsing Table

- To build a parsing table $M[A, a]$, for each $A \rightarrow \alpha$
 - $\forall a \in \text{FIRST}(\alpha): M[A, a] = A \rightarrow \alpha$
 - $\epsilon \in \text{FIRST}(\alpha) \Rightarrow \forall b \in \text{FOLLOW}(A): M[A, b] = A \rightarrow \alpha$

E	$\rightarrow$	$T E'$
E'	$\rightarrow$	$+ T E' \mid \epsilon$
T	$\rightarrow$	$F T'$
T'	$\rightarrow$	$* F T' \mid \epsilon$
F	$\rightarrow$	$(E) \mid \text{id}$

NON - TERMINAL	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow T E'$			$E \rightarrow T E'$		
E'		$E' \rightarrow + T E'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow F T'$			$T \rightarrow F T'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow * F T'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

- Choose the production according to the table, empty means error

Building a Parser in Practice

- Each non-terminal has a procedure or function for parsing
- => Recursive-Descent Parser
-  **Problem 1:** Backtracking may be necessary
 - when one derivation does not work, we may try others
-  **Problem 2:** A **left-recursive** grammar can cause **infinite loops**
 - when expanding a non-terminal, we may find itself and expand it again

LL(1) Grammar: Formal Definition

- LL(1) grammar (Not ambiguous! Not left-recursive!)
- A grammar is LL(1) if and only if any $A \rightarrow \alpha \mid \beta$ satisfies:
 - (1) For no terminal a do both α and β derive strings starting with a (by left factoring)
 - (2) At most one of α and β derive the empty string
 - (3) If $\beta \Rightarrow^* \epsilon$, α doesn't derive strings starting with terminals in FOLLOW(α)

E	$\rightarrow$	$T E'$
E'	$\rightarrow$	$+ T E' \mid \epsilon$
T	$\rightarrow$	$F T'$
T'	$\rightarrow$	$* F T' \mid \epsilon$
F	$\rightarrow$	$(E) \mid \text{id}$

Recursive Predictive Parsing

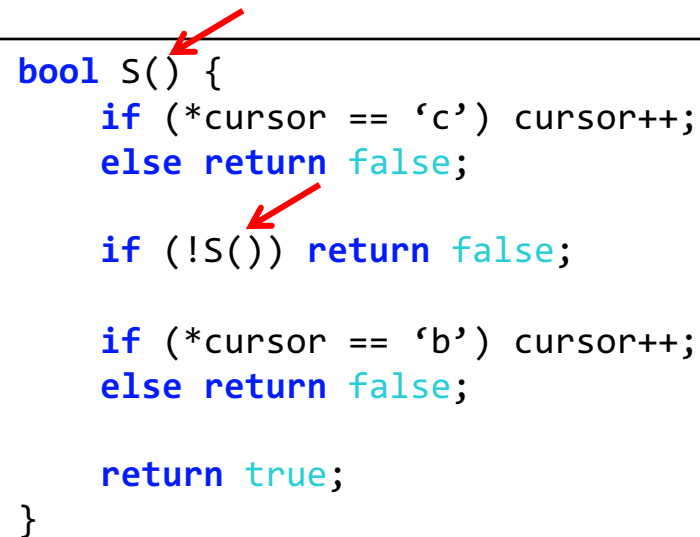
- Each non-terminal has a procedure or function for parsing
- => Recursive-Descent Parser
- **Example:** $S \rightarrow a S b \mid \epsilon$

```
bool S() {
    if (*cursor == 'c') cursor++;
    else return false;

    if (!S()) return false;

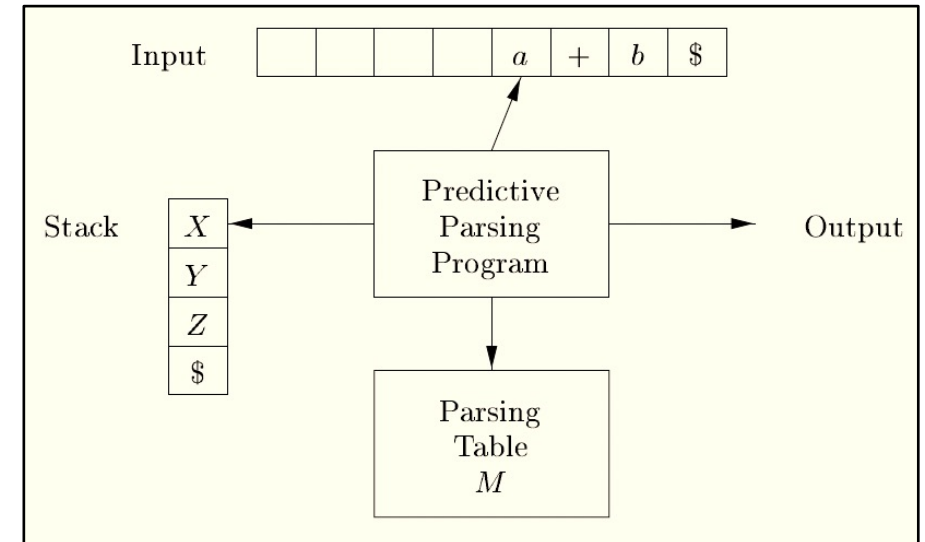
    if (*cursor == 'b') cursor++;
    else return false;

    return true;
}
```



Non-recursive Predictive Parsing

- How can we build a predictive parser without recursion?
- Maintain a stack explicitly!
- Initially, we put S in the stack
- When $A \rightarrow \alpha$ is applied, pop A , push α
- Building the PDA from CFG!
- Parsing table is the set of transition functions of PDA!



PART II: Bottom-Up Parsing

Bottom-up Parsing

- Keep shifting until we see the right-hand side of a rule
- Keep reducing as long as the tail of our shifted sequence matches the right-hand side of a rule. Then go back to shifting
- Once we have the right-hand side of a rule:
 - Do we reduce right away, or do we keep shifting more symbols?
 - What if there are multiple rules with the same RHS to reduce by?

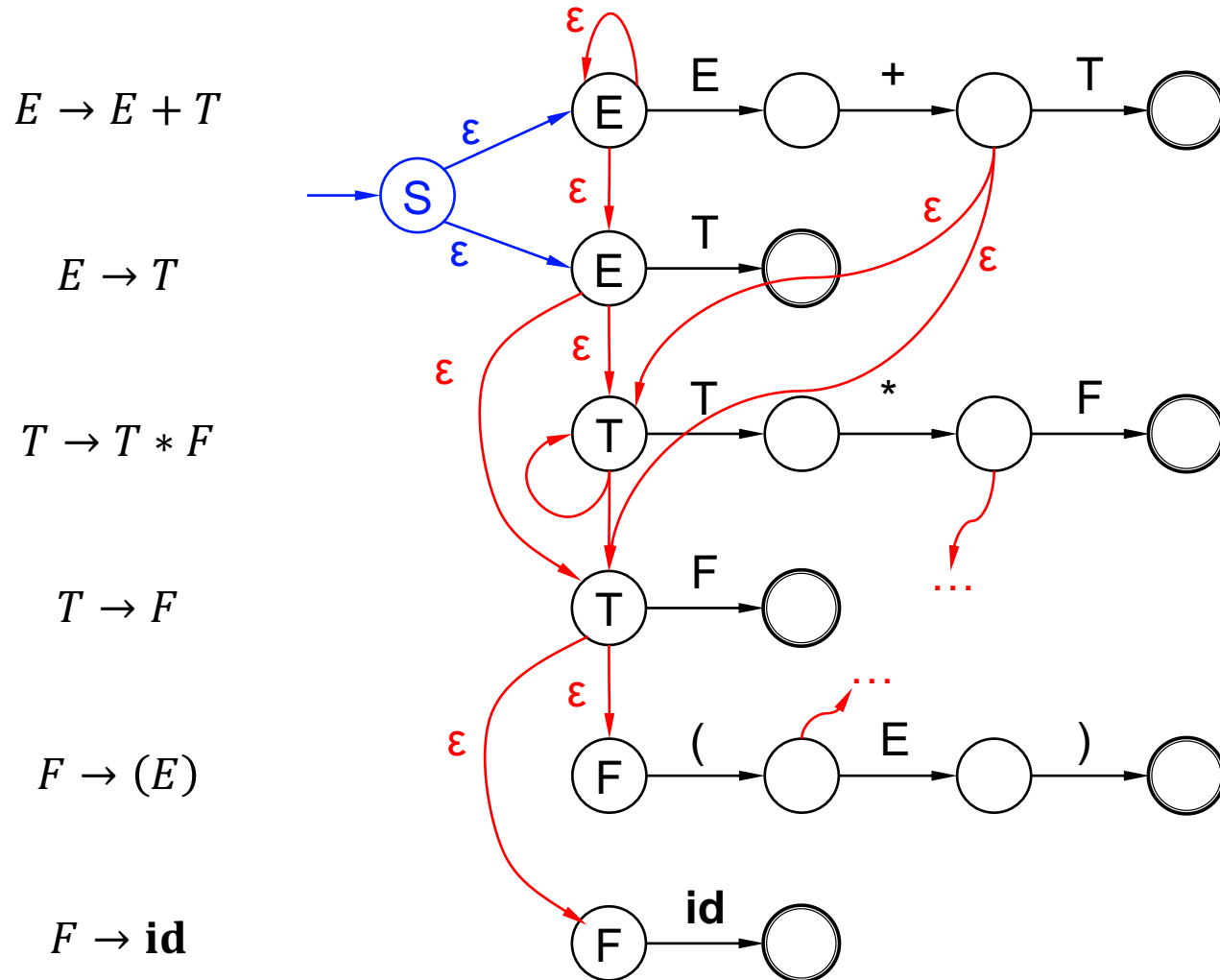
LR(0)

- LL(1) top-down parsing, we dealt with the tough decisions by just saying
 - "if we have to make decisions, it's not an LL(1) grammar".
- We'll start out by looking at LR(0) parsing
 - We only worry about how to handle grammars that **don't require us to make decisions during parsing**
 - Left-to-right scanning
 - Right-most derivation
 - Zero symbols of lookahead

LR(0)

- Keep shifting until we see the right-hand side of a rule
- Keep reducing as long as the tail of our shifted sequence matches the right-hand side of a rule. Then go back to shifting
- If this algorithm ever has to make decisions about which rule to
- reduce by, we give up and say "the grammar is not LR(0)".
 - LR(1), SLR(1), ...
 - Refer to Chapter 4, the Dragon book!

Recognizing the RHS



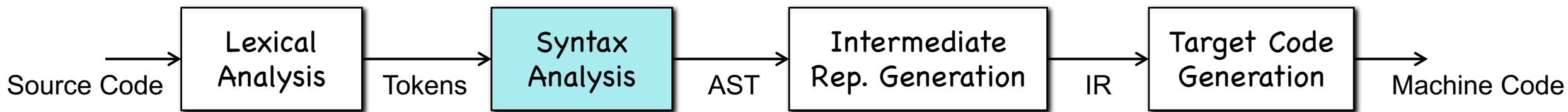
Create DFAs for the RHS of each rule and mark the initial states with the LHS.

For each state with a transition leading **outwards on a nonterminal**, connect the state (using ϵ -transitions) to all the states marked with that nonterminal.

We can transform it into a DFA

When shifting during LR(0) parsing, if we reach a final state, use the corresponding non-terminal to reduce.

Summary



- Syntax analysis is a procedure of building the parse/syntax tree
 - Top-down parsing and LL parsing
 - Recursive-descent parsers
 - Eliminating left-recursive
 - Predictive parsers, LL(1) parsers
 - Non-recursive predictive parser vs. PDA
 - Bottom-up parsing and LR parsing
 - LR(0) parser
 - Refer to § 4.5, Chapter 4, the Dragon book!

